Runtime Transform Gizmos - Quick Start Guide

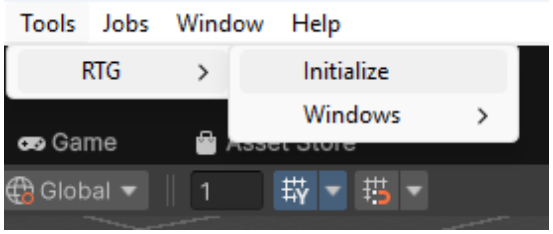
Support E-Mail: octamodius@yahoo.com

Support Forum: Join the Discussion

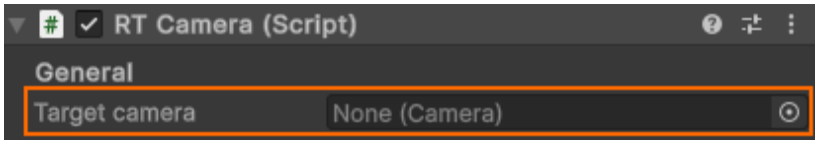
1. 🔧 Plugin Initialization

To get started, follow these basic setup steps:

1. **Import the Plugin** into your project via the Unity Package Manager.
2. Click on **Tools/RTG/Initialize**:



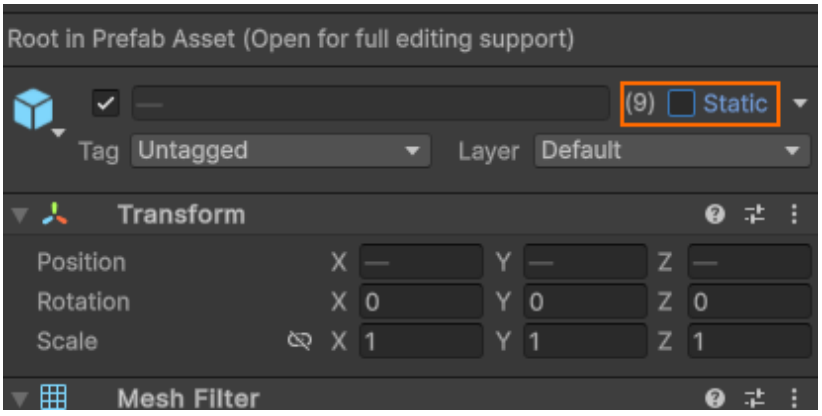
3. [Optional] If no camera is tagged **MainCamera**, assign one manually to the **Target Camera** field in the **RTCamera** object:



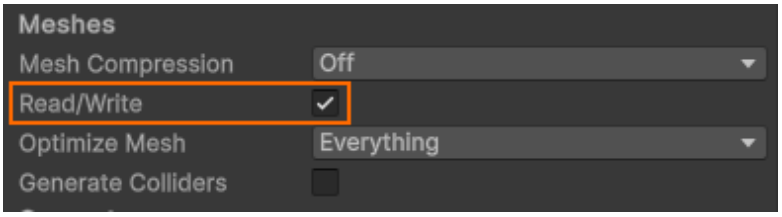
Scene Interaction Requirements

To interact with objects at runtime:

1. **Objects must be dynamic** (i.e., not marked as **Static**).
   Uncheck **Static** in the Inspector for all relevant prefab assets and scene objects:



2. **Meshes must be Read/Write enabled**:



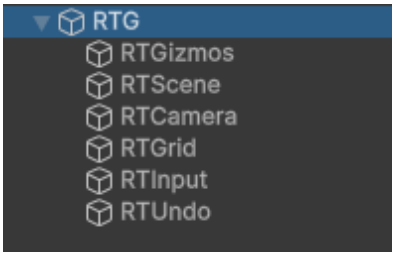
2. 📚 Scripting Tutorials

Example scripts are provided in the **Tutorials** folder. These tutorials should be followed in order:

1. **Tutorial_0_Creating_Gizmos** – Gizmo creation and initialization.
2. **Tutorial_1_Gizmo_Enable_Disable** – Enabling/disabling gizmos via code.
3. **Tutorial_2_Pivots_And_Transform_Spaces** – Working with pivots and transform spaces.
4. **Tutorial_3_IRTObjectTransformGizmoTarget** – Using the `IRTObjectTransformGizmoTarget` interface.
5. **Tutorial_4_Selection_Manager** – Implementing a runtime selection system.
6. **Tutorial_5_Multiple_Viewports** – Managing multiple camera viewports.

3. 🧩 Plugin Modules

After initialization, RTG creates a hierarchy of modular components:



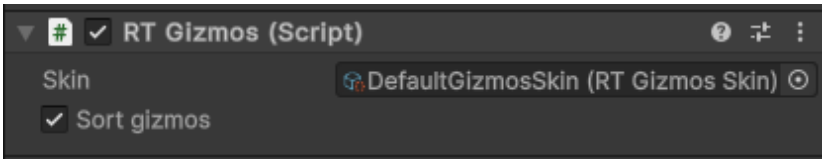
Each module serves a specific function. The **RTG** module is the core entry point and runtime initializer.

3.1 🧲 RTGizmos Module

Handles creation, rendering, and interaction of gizmos. For example:

```
RTGizmos.get.CreateObjectMoveGizmo();   // Create an object move gizmo
```

Inspector Overview



1. **Sort Gizmos**: Ensures correct draw order (important because gizmos use no depth testing).
2. **Skin**: Manages visual styles and behavior like snapping, hover padding, etc.

A default skin is located in `Resources/Gizmos/Skins`.
To create a new one:

1. Right-click in the **Project** view.
2. Click on **Create/RTG/RTGizmosSkin**.

Clicking on a skin asset shows a toolbar:

Each button opens settings for a specific gizmo type. The **Global** button opens global settings shared by all gizmos.

3.2 🌐 RTScene Module

The **RTScene** module manages runtime scene objects and provides raycasting and spatial queries. Because Unity does not notify this system of changes automatically, you must manually register, update, and unregister objects using the following API calls:

Registering Objects

```
RTScene.get.RegisterObjectHierarchy(gameObject); // Call this when you create an object
```

Unregistering Objects

```
RTScene.get.UnregisterObjectHierarchy(gameObject); // Call this before destroying an object
```

After Adding Components

```
RTScene.get.OnObjectComponentsChanged(gameObject);
```

After Transform Changes

```
RTScene.get.OnObjectTransformChanged(gameObject);
```

Alternatively, you can also change an object's transform using these functions, which call `OnObjectTransformChanged` behind the scenes:

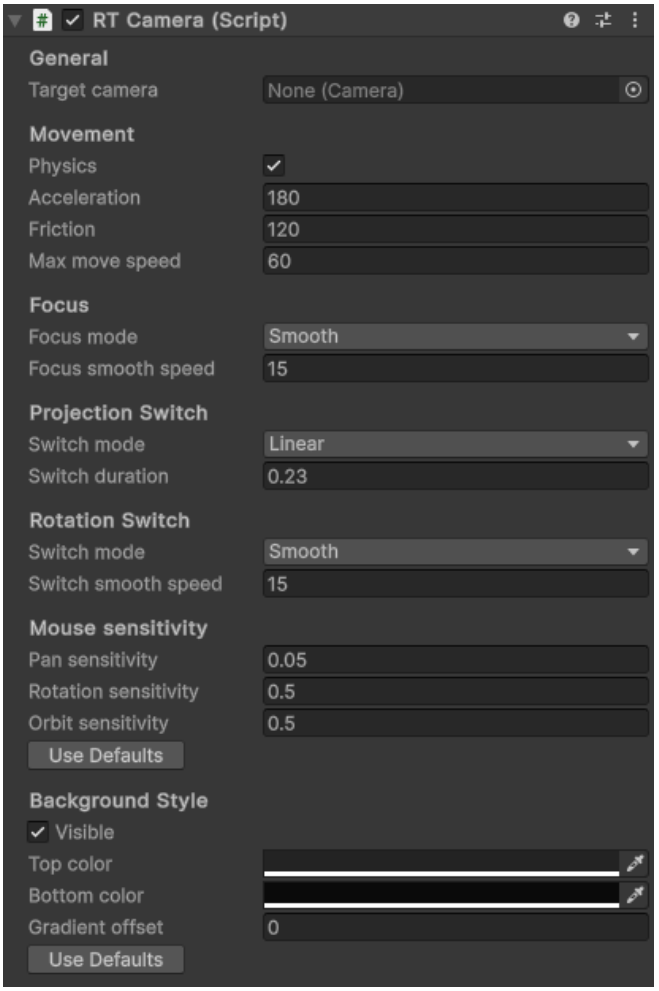
```
RTScene.get.SetObjectTRS(gameObject, position, rotation, scale);
RTScene.get.SetObjectPosition(gameObject, position);
RTScene.get.MoveObject(gameObject, moveOffset);
RTScene.get.SetObjectRotation(gameObject, rotation);
RTScene.get.RotateObject(gameObject, rotation);
RTScene.get.RotateObjectAroundPivot(gameObject, rotation, pivot);
RTScene.get.SetObjectScale(gameObject, scale);
```

These calls ensure efficient tracking without resorting to continuous polling, which would otherwise introduce unnecessary overhead.

3.3 🎥 RTCamera Module

Represents the active scene camera and controls view navigation. The active scene camera is the camera that interacts with the gizmos.

Inspector Overview



- **Navigation**: Pan, rotate, orbit sensitivity, focus behavior.
- **Projection Switch**: Controls transition between perspective and orthographic.
- **Rotation Switch**: Handles view alignment when clicking the Scene Gizmo cones.

Switching Cameras

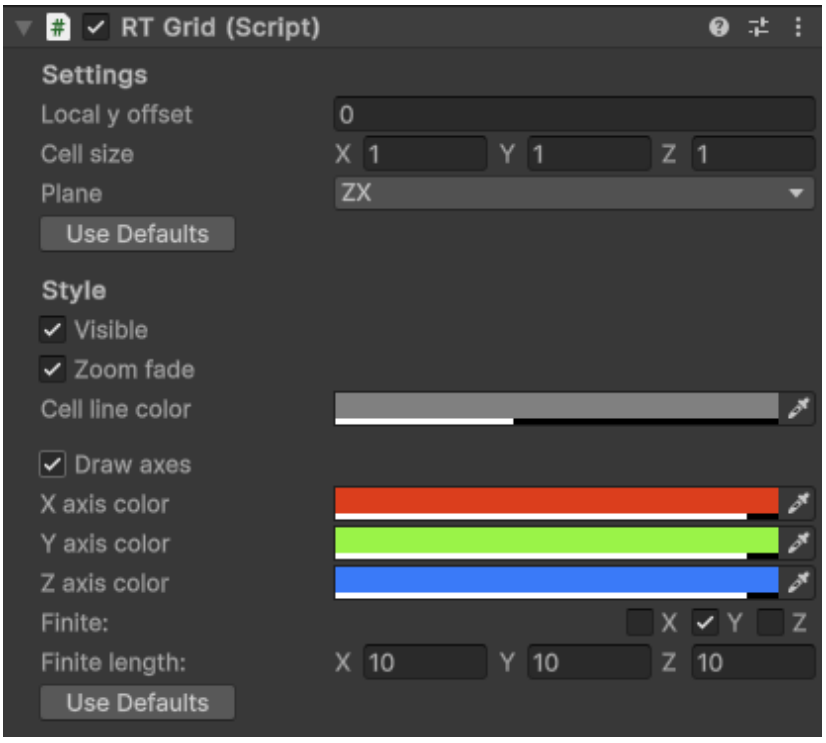
```
RTCamera.get.settings.targetCamera = camera_TopLeft;
```

Useful for multi-viewport setups where you might wish to switch between different viewports.

3.4 📐 RTGrid Module

Defines a 3D scene grid used for movement snapping and vertex snapping.

Inspector Overview



1. At the top of the UI, you'll find the following grid settings:
    1. **Local Y Offset** – Controls the grid's offset along its local Y axis. By default, this aligns with the world Y axis, but it may change depending on the selected **Plane**.
    2. **Cell Size** – Sets the size of each grid cell. Although the grid appears flat in the scene, it is a 3D construct, so the cell size is defined as a 3D vector.
    3. **Plane** – Determines the grid's orientation in the scene. For example, when working with 2D sprites, you may want to set this to **XY**.

    At the bottom, style settings let you adjust the visual appearance of the grid.

    By default, you can move the grid up or down using the **]** and **[** keys.

    Holding the **[G]** key activates **Grid Action Mode**, enabling mouse-driven grid adjustments:

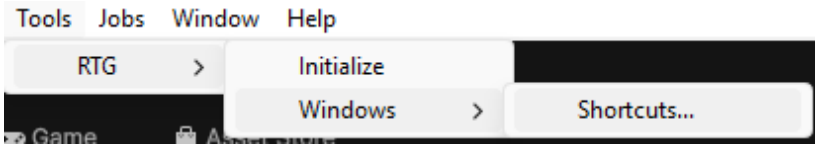
    1. **Left-click** – Snaps the grid to the cursor's pick point.
    2. **Right-click** – Snaps the grid above or below the bounds of the picked object (based on its volume).

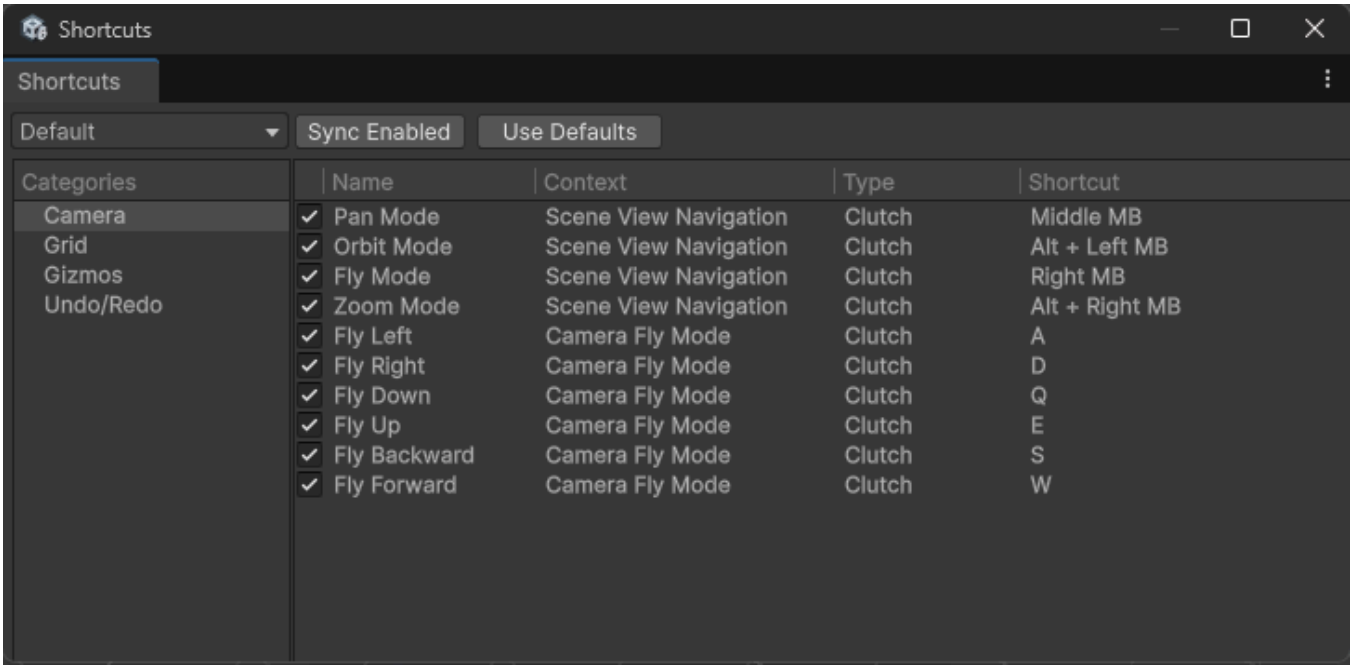
3.5 ⌨️ RTInput Module

Handles input and shortcut management.

Shortcuts Window

The plugin allows you to configure shortcuts using the shortcuts window:





Shortcuts are organized into categories listed on the left. When you select a category, the right panel displays all shortcuts within that category.

To change a shortcut binding, double-click the text in the **Shortcut** column (right panel), then press the desired key combination.

The toggles in the first column let you enable or disable individual shortcuts.
Note: Changes only apply to the currently active profile. To apply the enabled/disabled states to other profiles, click the **Sync Enabled** button.

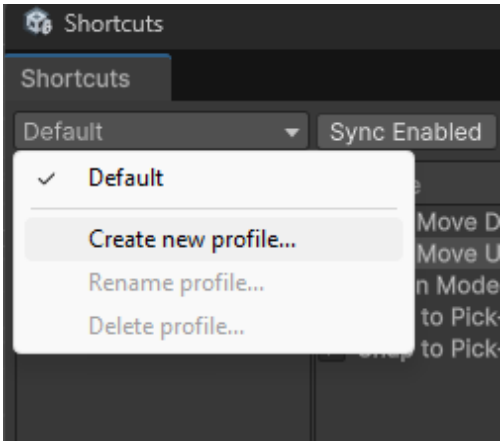
Each shortcut is associated with a **context**, which defines when and under what conditions the shortcut's command is triggered. Two or more shortcuts that share the same context and have the same binding are said to be in conflict. Shortcuts that are part of a conflict will display a small **warning** icon on the far right, next to the binding.

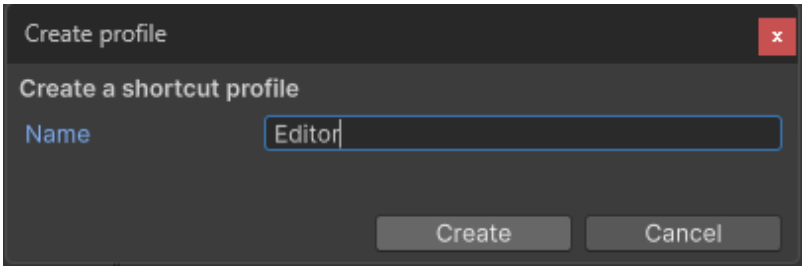
3.5.2 Avoiding Shortcut Conflicts with Unity Editor

RTG implements Undo/Redo support using the same shortcut keys as Unity (**Ctrl + Z** / **Ctrl + Y**). This means that when you test your app inside the Unity Editor, pressing **Ctrl + Z** will also trigger an undo in the Unity Editor itself —often leading to unexpected or annoying behavior.

To avoid this conflict, you can set up a custom shortcut profile. Here's how:

1. Create a new shortcut profile. You can name it something like **Editor**:





2. **Navigate to the Undo/Redo category** in the shortcut editor.
3. **Change the bindings**:
   - Set **Undo** to `Ctrl + Shift + Z`
   - Set **Redo** to `Ctrl + Shift + Y`

✅ While developing your app, switch to the **Editor** profile to avoid conflicts with Unity's shortcuts.
📦 For final builds, switch back to the **Default** profile so that end users get the familiar `Ctrl + Z / Y` behavior.

3.5.3 Creating Custom Shortcuts

Sometimes you may need shortcuts for custom functionality not available by default. However, you can't define new shortcuts directly in the UI, as each shortcut requires a **Context** and a **Command**, which must be defined in code.

While you *could* hardcode your own key input logic, doing so outside the plugin's shortcut system increases the risk of conflicts and inconsistency.

Example: Toggling Move Snap Mode with `H`

Let's say you want to bind the `H` key to toggle between **Relative** and **Absolute** move snapping modes:

1. Create a new C# script called **CustomCommands.cs**. You can create it anywhere you wish and give it any name you like.
2. Create a class as shown below:

```
namespace RTGBasic
{
    [Serializable] public class Command_Gizmos_ToggleMoveSnapMode : ActionCommand
    {
        protected override void OnActivate()
        {
            EGizmoMoveSnapMode newSnapMode = RTGizmos.get.moveSnapMode ==
EGizmoMoveSnapMode.Relative ?
                EGizmoMoveSnapMode.Absolute : EGizmoMoveSnapMode.Relative;
            RTGizmos.get.moveSnapMode = newSnapMode;
        }
    }
}
```

3. Open up the **ShortcutProfile.cs** file inside the **Scripts/Input** folder.
4. Locate the **RefreshShortcuts** function.
5. At the end of the function add the following code:

```
//-----------------------------------------------------------------------------
// Custom Shortcuts
//-----------------------------------------------------------------------------
// Note: If you ever want to change the command of a shrotcut or its context, you will need
//       to re-initialize the plugin for the changes to take effect.
sc = FindOrCreateShortcutCategory(ShortcutCategoryNames.gizmos);
sc.FindOrCreateShortcut("Move Snap Mode",   new Command_Gizmos_ToggleMoveSnapMode(),
 ctx_SceneView, "h");
```

Understanding Shortcut Contexts

Shortcut contexts determine **when** a shortcut is allowed to fire. The `ctx_SceneView` is an instance of `Context_SceneView` and is defined in `Contexts.cs`:

```
//-----------------------------------------------------------------------------
// Name: Context_SceneView (Public Class)
// Desc: Scene view context in which the user can change what happens in the scene
//       (e.g. camera navigation, grid etc).
//-----------------------------------------------------------------------------
[Serializable] public class Context_SceneView : Context
{
    public Context_SceneView() : base("Scene View") { }
    public override bool IsActive() { return !RTScene.get.IsUGUIHovered(); }
}
```

This means the shortcut will only be active when the mouse is **not** hovering over a UI element. You can create your own contexts by following this template.

> ⚠️ **Important:**
> If you change a shortcut's command or context, you must re-initialize the plugin for the changes to take effect.

> 💾 **Tip:**
> Keep a local backup of your modified `ShortcutProfile.cs`. Plugin updates may overwrite it, and you'll need to restore your changes manually.

Action vs Clutch Commands

There are two types of shortcut commands:

1. **Action – Executes once** when the key is pressed.
2. **Clutch – Executes and stays active** while the key is held down**.

Example: Clutch Command for Vertex Snapping Defined by RTG

```
//-----------------------------------------------------------------------------
// Name: Command_Gizmos_VertexSnap (Public Class)
// Desc: Command which toggles vertex snapping on/off.
//-----------------------------------------------------------------------------
[Serializable] public class Command_Gizmos_VertexSnap : ClutchCommand
{
    protected override void OnActivate() {
RTGizmos.get.SetMoveGizmosMoveMode(EGizmoMoveMode.VertexSnap); }
    protected override void OnDeactivate() {
RTGizmos.get.SetMoveGizmosMoveMode(EGizmoMoveMode.Normal); }
}
```

3.6 ↩ RTUndo Module

The **RTUndo** module implements Undo/Redo functionality and exposes a setting in the Inspector to configure the size of the undo/redo stack.